

Greedy Algoritmaları

Greedy Paradigması

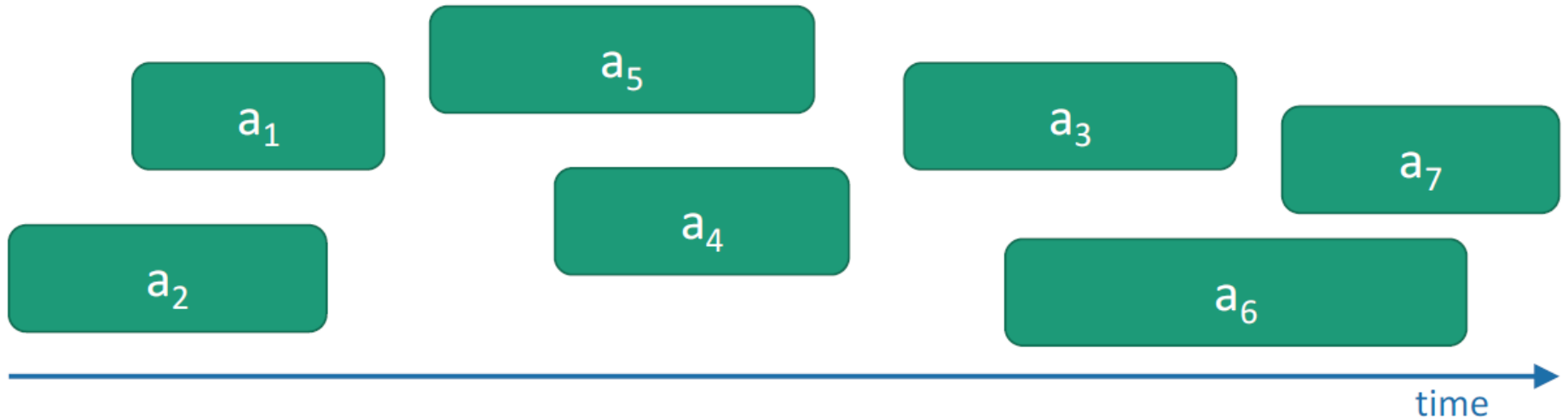
- *Greedy algoritma tasarım tekniği her zaman o an için en iyi gözüken seçimi yapar.*
 - Yerel olarak en uygun seçim yapılarak küresel olarak en uygun çözüme ulaşılması hedeflenmektedir.
 - Bazı problemler için bu hedef gerçekleşebilir.
- Dinamik programlama paradigmasına göre kodlanması kolaydır.

Etkinlik Seçimi (Activity-Selection) Problemi

- **Problem:** Bir etkinlik dizisinden maksimum düzeyde yararlanmak
 - *Activity-Selection Problem*

- Input:
 - Activities $a_1, a_2, \dots, a_n$
 - Start times $s_1, s_2, \dots, s_n$
 - Finish times $f_1, f_2, \dots, f_n$
- Output: Bugün kaç aktivite yapabilirsiniz?

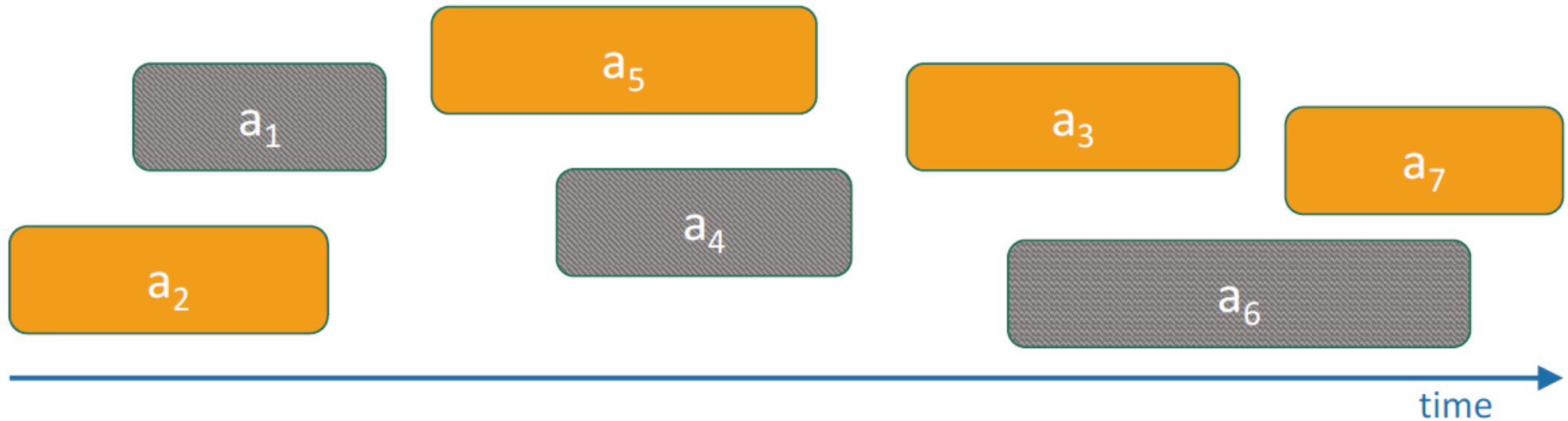
Greedy Algorithm



«Karşılıklı uyumlu faaliyetlerin» maksimum büyüklükteki A alt kümesini bulunuz.

Faaliyetlerde harcanan maksimum zaman değil, maksimum aktivite sayısı

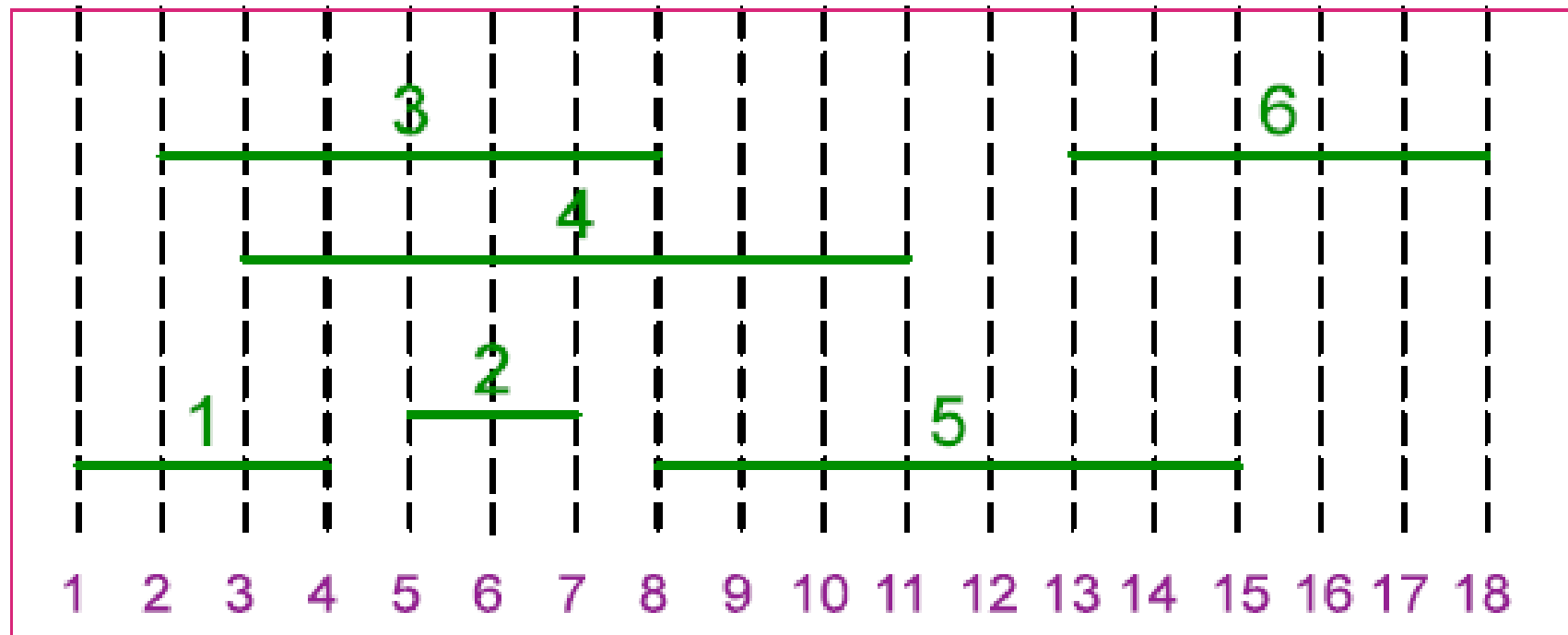
Ekleyebileceğin «en küçük bitiş süresine sahip olan aktiviteyi» seç



- Running time: $O(n)$ Eğer aktiviteler zaten bitiş zamanına göre sıralı ise
- Running time: $O(n \log(n))$ Bitiş zamanına göre sırlamayı biz yaparsak
 $O(n) + O(n \log(n)) = O(n \log(n))$

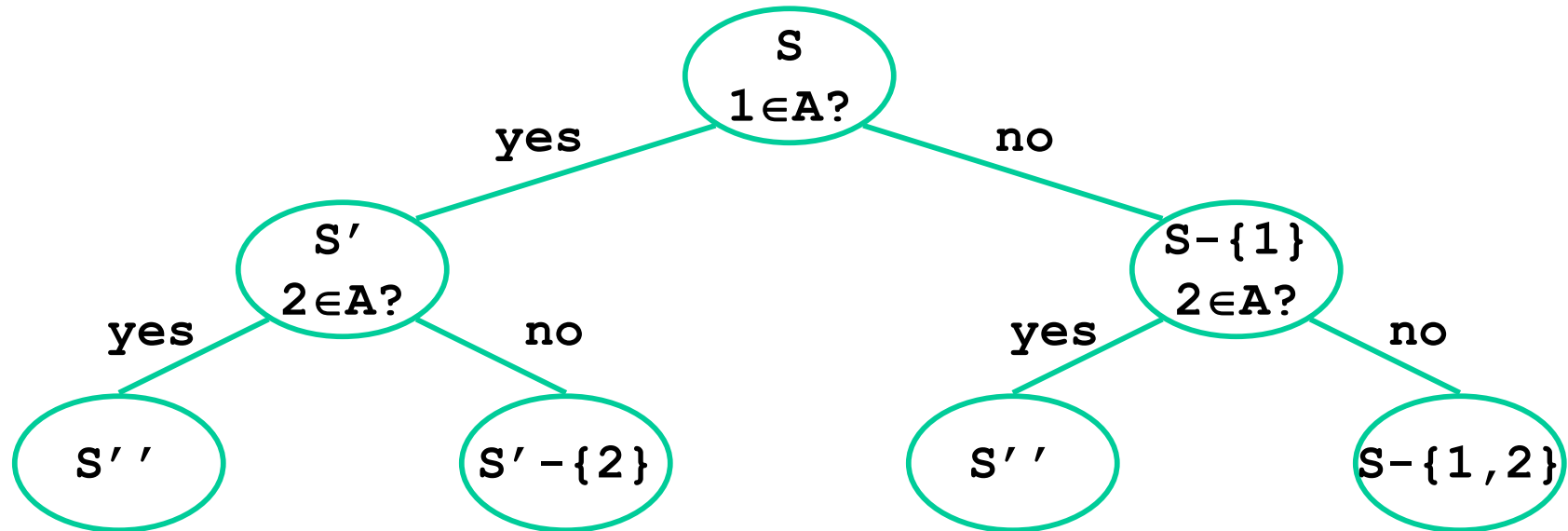
Etkinlik Seçimi (Activity-Selection) Problemi-Örnek

$$S = \{[1, 4), [5, 7), [2, 8), [3, 11), [8, 15), [13, 18)\}$$



Optimal Altyapı (Substructure)

Theorem: $k, A \subseteq S$ optimal çözümünde en erken bitiş süresine sahip aktivite olsun, o zaman $A - \{k\}$ alt problem için optimal bir çözümdür.



Etkinlik Seçimi: Greedy Çözüm

- Aktiviteler bitiş saatine göre sıralayın
- İlk aktiviteyi planlayın
- Daha sonra, önceki aktivite bittikten sonra başlayacak şekilde sıralanmış listedeki bir sonraki aktiviteyi planlayın
- Eklenecek aktivite kalmayana kadar tekrarlayın
- Sezgi daha basittir: Her zaman o anda mevcut olan en kısa mesafe seçilir.

GAS(s, f, n)

$A \leftarrow \{1\}$

$j \leftarrow 1$

for $i \leftarrow 2$ **to** n **do**

if $s_i \geq f_j$ **then**

$A \leftarrow A \cup \{i\}$

$j \leftarrow i$

return A

Running time: $\Theta(n)$

Etkinlikler zaten sıralı ise

Örnek

Devam

$S = \{[1, 4), [5, 7), [2, 8), [3, 11), [8, 15), [13, 18)\}$

$\rightarrow A = \{1, 2, 5\}$

GAS(s, f, n)

$A \leftarrow \{1\}$

$j \leftarrow 1$

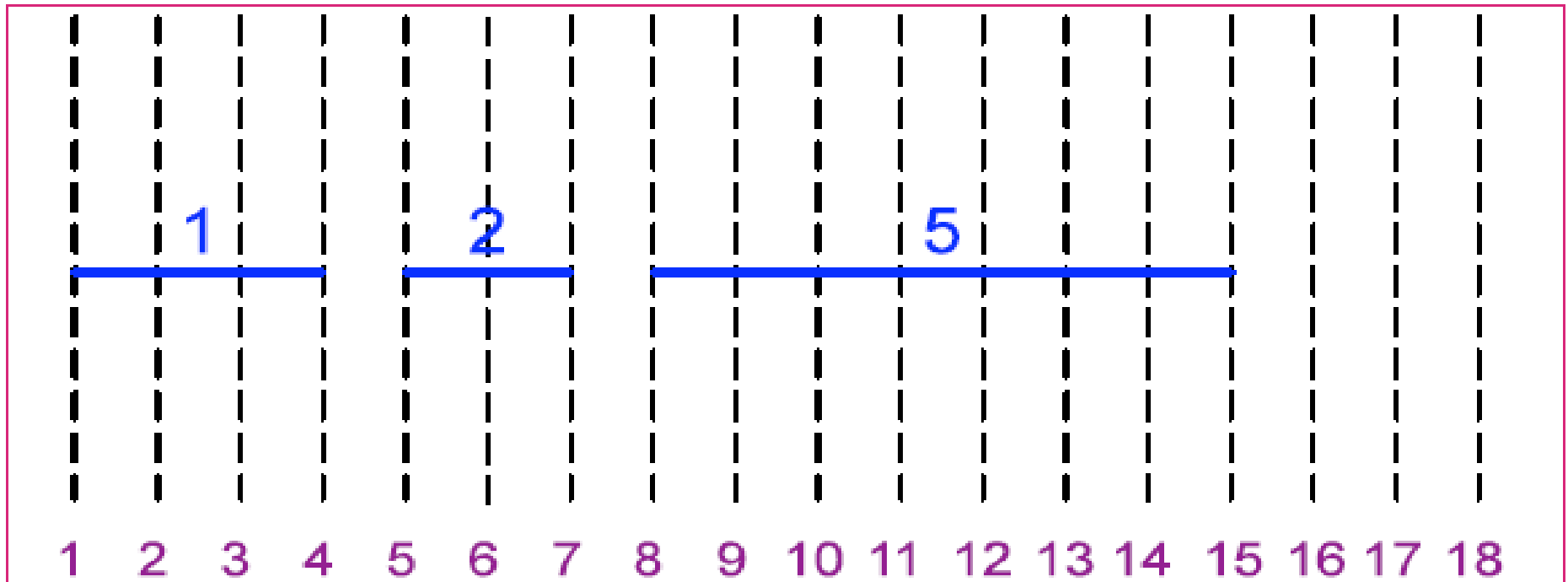
for $i \leftarrow 2$ to n do

 if $s_i \geq f_j$ then

$A \leftarrow A \cup \{i\}$

$j \leftarrow i$

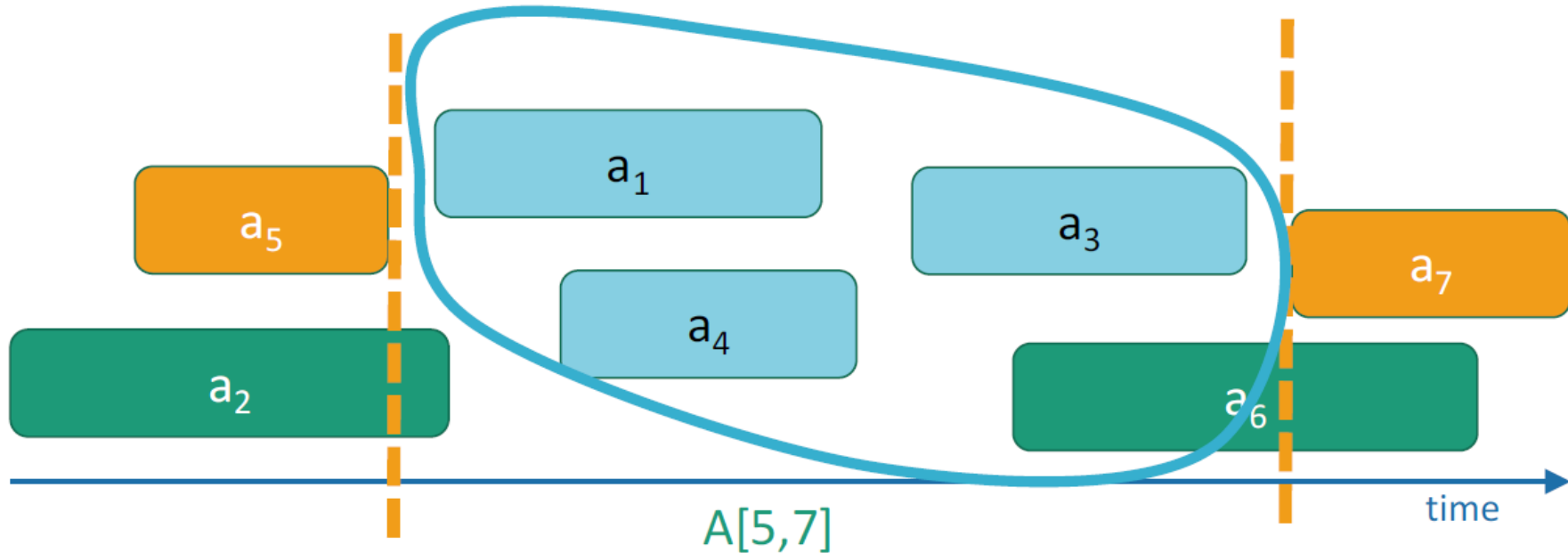
return A



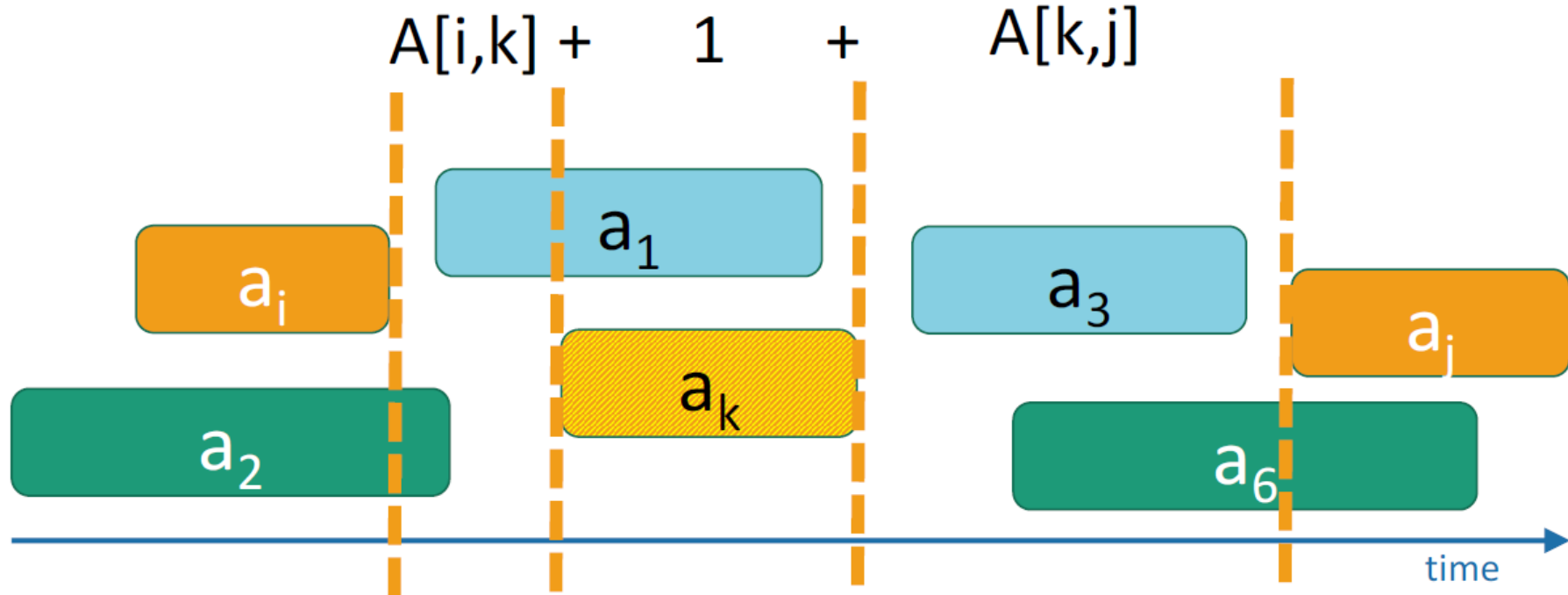
Etkinlik Seçimi Problemi Dinamik Programlama

Optimum alt yapıyı tanımlayalım

$A[i,j]$ = Aktivite i bittikten sonra ve Aktivite j başlamadan önce sıkıştırılabileceğiniz aktivite sayısı



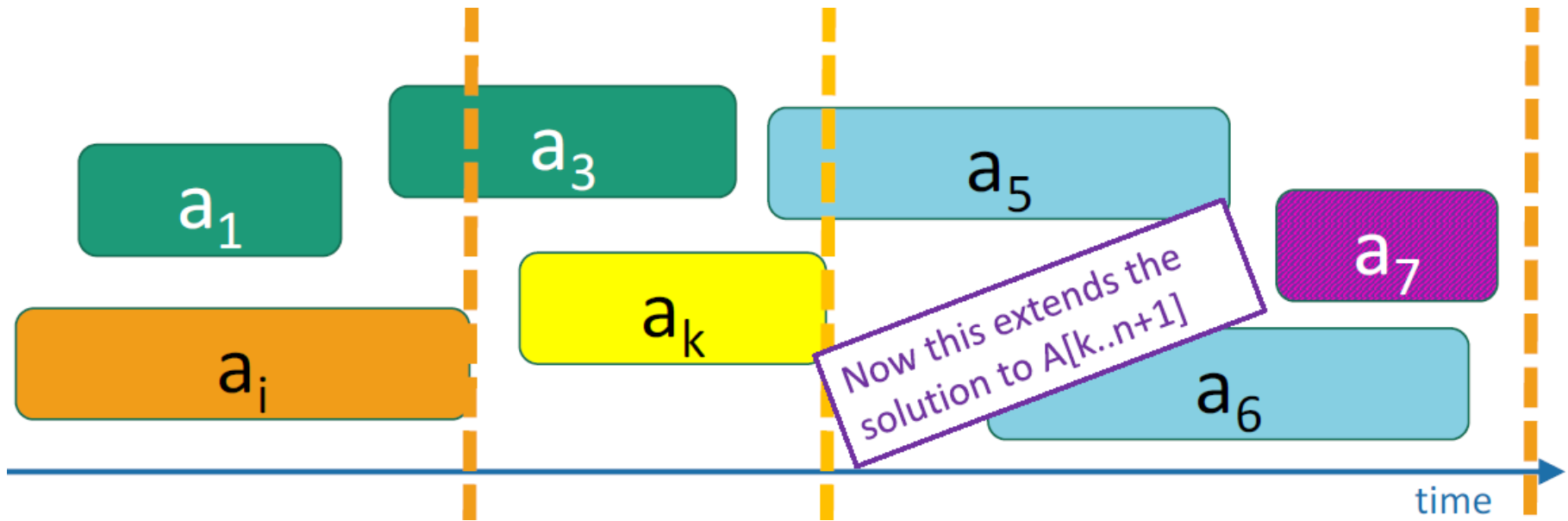
$$A[i,j] = \max_k \{ A[i,k] + 1 + A[k,j] \}$$



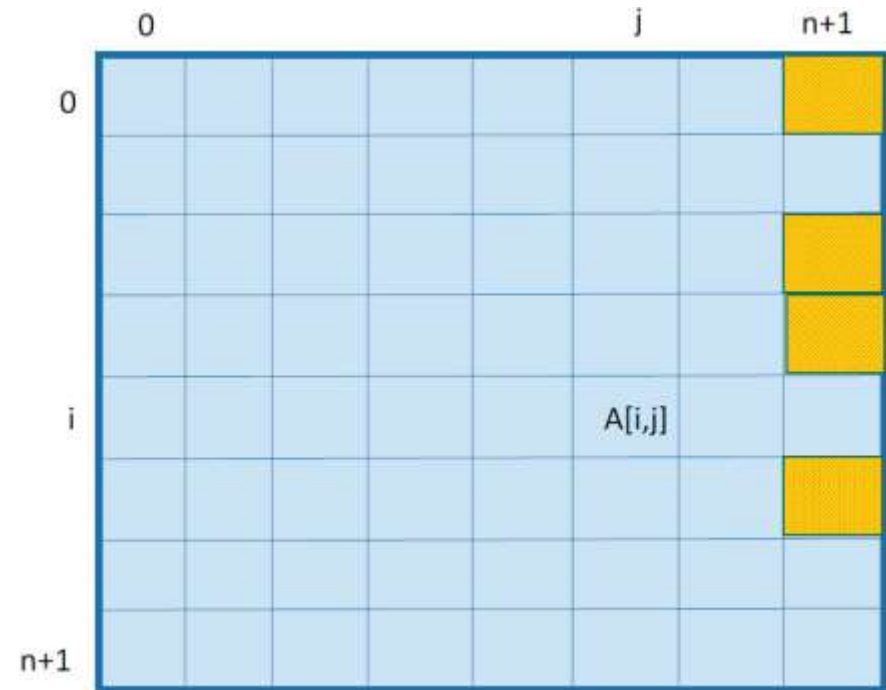
Diyelim ki k , i 'den sonra en kısa bitirme süresiyle sıkıştırılabileceğiniz aktivitedir.

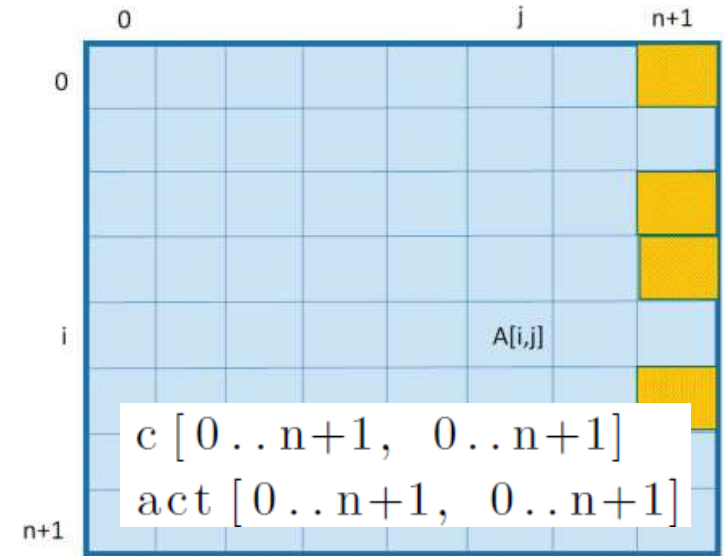
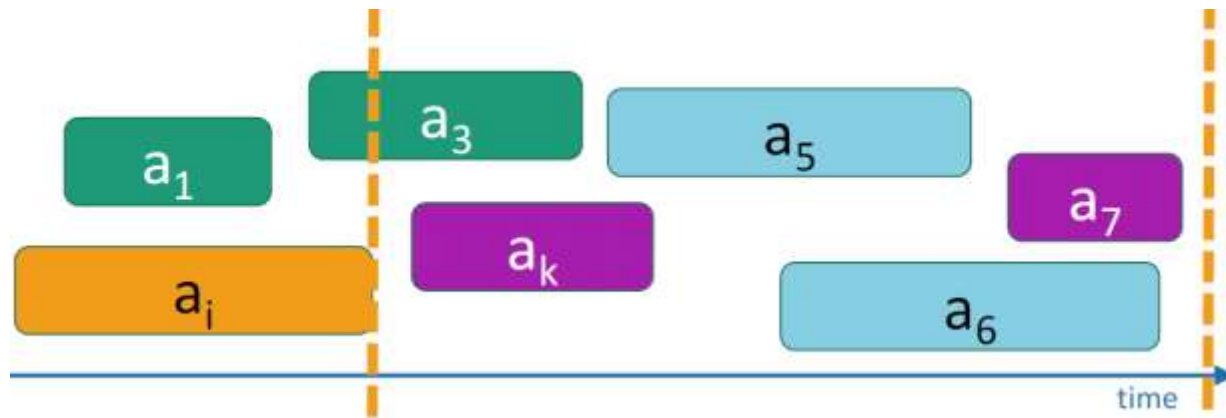
O zaman $A[i..n+1]$ için optimal bir çözüm vardır ki bu da optimal çözümü $A[k..n+1]$ 'e genişletir.

Running time: ?



$A[0, n+1]$ istenen değerdir ve sadece önceden Hesaplanan ($A[2, n+1]$) değerlere bağlıdır.





DYNAMIC-ACTIVITY-SELECTOR(s, f, n)

```

for  $i = 0$  to  $n$ 
     $c[i, i] = 0$ 
     $c[i, i+1] = 0$ 
 $c[n+1, n+1] = 0$ 

for  $l = 2$  to  $n+1$ 
    for  $i = 0$  to  $n-l+1$ 
         $j = i+1$ 
         $c[i, j] = 0$ 
         $k = j - 1$ 

```

```

    while  $f[i] < f[k]$ 
        if  $f[i] \leq s[k]$  and  $f[k] \leq s[j]$ 
            and  $c[i, k] + c[k, j] + 1 > c[i, j]$  do
                 $c[i, j] = c[i, k] + c[k, j] + 1$ 
                 $act[i, j] = k$ 
         $k = k - 1$ 

```

"Karşılıklı olarak uyumlu faaliyetlerin maksimum boyutu"

```

print  $c[0, n+1]$ 
print "The set contains "
PRINT-ACTIVITIES( $c, act, 0, n+1$ )

```

Hangi etkinlikler?

```
PRINT-ACTIVITIES(c, act, i, j)
    if c[i, j] > 0
        k = act[i, j]
        print k
    PRINT-ACTIVITIES(c, act, i, k)
    PRINT-ACTIVITIES(c, act, k, j)
```

PRINT-ACTIVITIES prosedürü, en iyi çözüm A_{ij} 'ye yerleştirilen etkinlik kümesini yinelemeli olarak yazdırır. İlk olarak, $C[i; j]$ 'nin maksimum değerini elde eden **k** etkinliğini yazdırır.

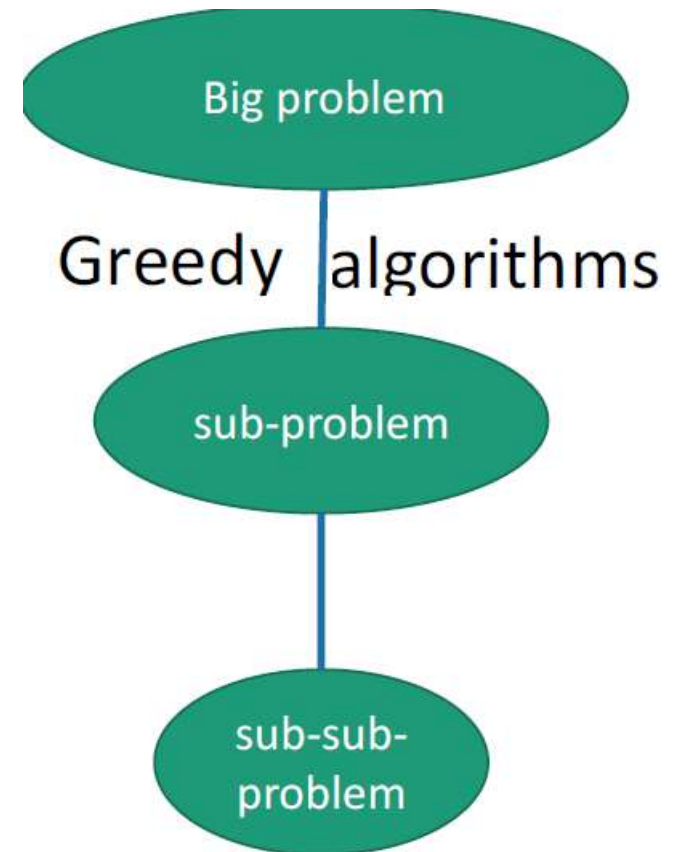
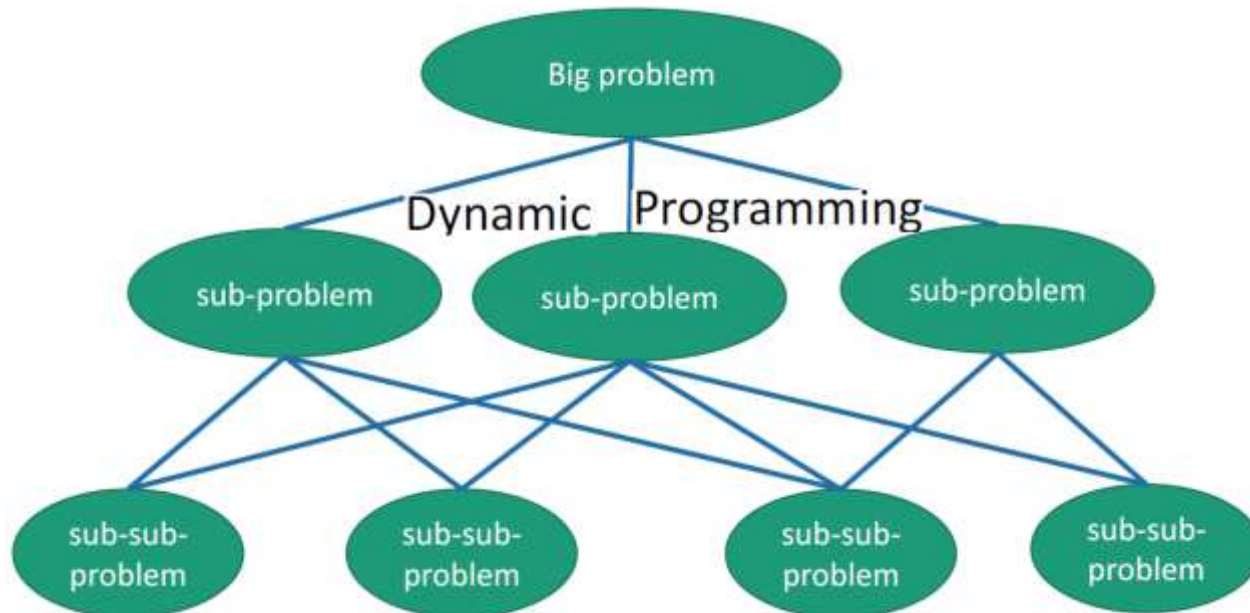
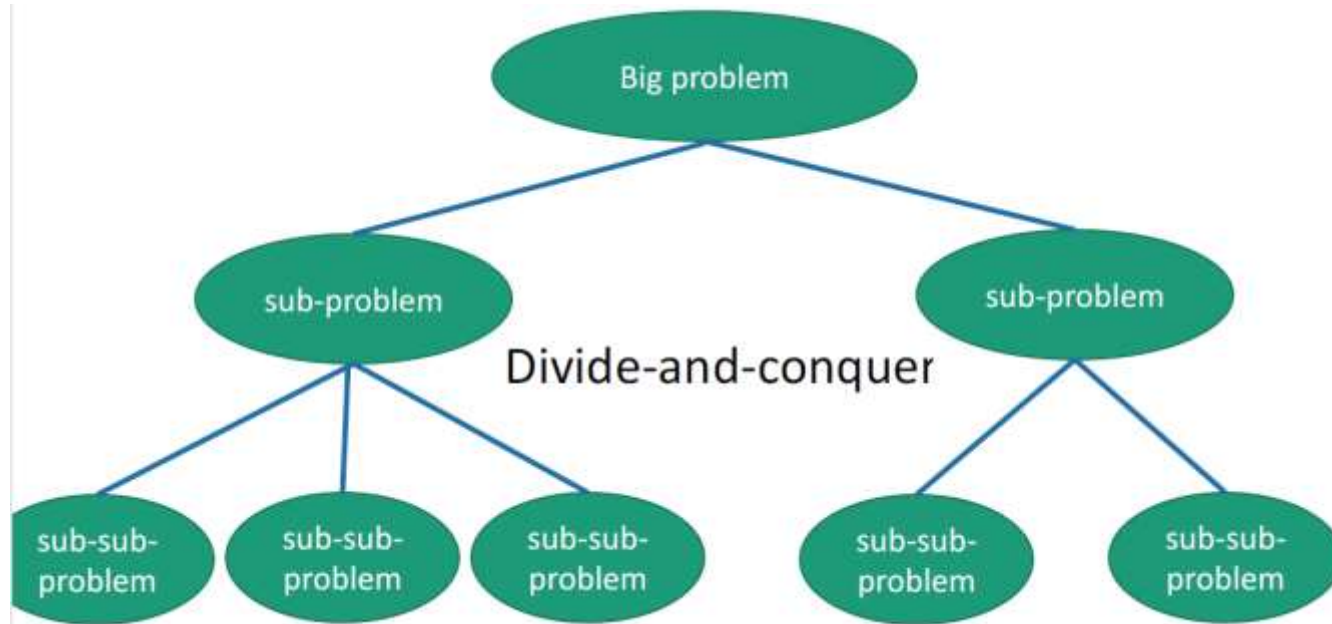
Daha sonra, $A(i, k)$ ve $A(k, j)$ 'deki etkinlikleri yazdırmak için yineleme yapar.

Yineleme, $c[i; j] = 0$ olduğunda en alta iner, böylece $A(i, j)$ =boş küme olur.

Dinamik Programlama	$O(n^3)$
Greedy	$\Theta(n)$

Greedy vs Dynamic Programming

- Hem Açıgözlü hem de DP stratejileri tarafından kullanılan optimum alt yapı özelliği
- **Greedy Seçimi:** Yerel olarak en iyi seçimlerin dizisi $\Rightarrow$ bir en iyi çözüm
 - Şu anda en iyi görünen seçimi yapıyoruz
 - Daha sonra seçim yapıldıktan sonra ortaya çıkan alt problemi çözüyoruz
- **DP:** Her adımda bir seçim/karar var. Ancak seçim alt problemlere yönelik en iyi çözümlere bağlı olabilir.
- **Greedy:** Seçim, şimdiye kadar yapılan seçimlere bağlı olabilir, ancak gelecekteki seçimlere veya alt problemlerin çözümlerine bağlı olamaz.
- DP aşağıdan yukarı (bottom-up) bir strateji kullanır.
- Greedy ise yukarıdan aşağı (top-down) strateji kullanır.
 - her açığözlü seçim, her alt problemi yinelemeli olarak benzer ancak daha küçük bir probleme indirger



Açgözlü algoritmalar ve dinamik programlama benzerdir; her ikisi de genel olarak aynı koşullar altında çalışır, ancak dinamik programlama öncelikle alt problemleri çözer.

0-1 sırt çantası problemini düşünün.

Bir hırsız 1..n. değerinde eşyaları olan bir mağazayı soyuyor. Her eşya $v[i]$ dolar değerinde ve $w[i]$ pound ağırlığında. Hırsız en fazla ganimet almak istiyor ancak sırt çantası yalnızca W ağırlığında. Hangi eşyaları almalı?

Açgözlü algoritma: Önce en değerli öğenin mümkün olduğunca çoğunu al. Mutlaka en iyi değeri vermez!

Kesirli sırt çantası problemini

düşünün. Bu sefer hırsız nesnelerin herhangi bir kısmını alabilir. Örneğin, altın külçe altın yerine altın tozu olabilir. Bu durumda, hırsızın taşıyabileceği ağırlık (değer/ağırlık) başına en değerli nesneden olabildiğince fazlasını alması, ardından bir sonraki değerli nesneden olabildiğince fazlasını alması ve daha fazla ağırlık taşıyamayacağı noktaya kadar devam etmesi gerekir.

Açgözlü algoritma bazen probleme bağlı olarak en iyi çözümü verir, bazen vermez.

Uygulanabilir olduğunda dinamik programlama genellikle **en iyi çözümleri** verir, ancak genellikle bulunması daha zordur ve bazen uygulanması daha zordur.

Böl ve yönet, tüm alt problemler bağımsız olduğunda en iyi şekilde çalışır. Bu nedenle, algoritmayı en verimli hale getiren bölümü seçin ve tüm problemi çözmek için çözümleri basitçe birleştirin.

Alt problemler birbirine bağımlı olduğunda, problemi nereye böleceğimizi bilmediğimizde dinamik programlamaya ihtiyaç duyulur.

Böl ve yönet, “çatışan alt problemlerle” karşılaşılmadığı durumlar için en uygundur.

Dinamik programlama algoritmalarında, genellikle her alt problemi yalnızca bir kez çözeriz ve çözümlerini saklarız. Ancak bu, alan extra bellek maliyetine yol açar.

Dinamik Programlama alt problemleri aşağıdan yukarıya çözer. Alt problemlerin tüm çözümlerini bulana kadar problem çözülemez. Çözüm, tüm problem ortaya çıktığında ortaya çıkar.

Greedy alt problemleri yukarıdan aşağıya çözer. Önce bir problem için açgözlü seçeneği bulmamız, sonra problemi daha küçük bir probleme indirgememiz gerekir. Çözüm, tüm problem ortadan kalktığında elde edilir.

Dinamik Programlama, problemi çözmeden önce her olasılığı denemek zorundadır. **Açgözlüden çok daha maliyetlidir.** Bununla birlikte, açgözlü yaklaşımın çözemediği ancak dinamik programlamanın çözebildiği bazı problemler vardır. Bu nedenle, önce açgözlü algoritmayı deneriz. Başarısız olursa, dinamik programlamayı deneriz.